

Setup and Deployment of ProtO-RU: A Software-Based O-RAN Split-7.2 Radio Unit

Devagya CS25MTECH12003

Project Recap

- Problem statement
 - To deploy, configure, and validate ProtO-RU an open-source software implementation of an O-RAN split-7.2 Radio Unit using standard PC hardware and commercial Software-Defined Radios (SDRs) to create a flexible, accessible testbed.
- Motivation
 - Commercial O-RUs are typically expensive, proprietary, and fixed-function hardware, which severely limits customisation and accessibility for independent 5G research. ProtO-RU addresses this by implementing a fully open-source software-defined O-RU on commodity CPUs and SDRs, removing the last closed-source dependency in an otherwise open 5G stack.
- Key Objectives
 - Successfully install ProtO-RU and its dependencies via Docker on a Linux host
 - Interface the software RU with the srsRAN DU and initiate the fronthaul (eCPRI/OFH) connection
 - Run end-to-end setup with hardware: UE → B210 → ProtO-RU → srsRAN → Open5GS
 - Document Split-8 baseline comparison

Related Work / Positioning

Related works:

- ProtO-RU - Open-source O-RAN Split-7.2 Radio Unit implementation using SDRs.
- srsRAN Project - Open-source 4G/5G RAN stack. Common SDR setup uses direct UHD connection to B210, similar to Split-8.
- Open5GS- Open-source 5G core used for AMF, SMF, UPF, and subscriber management.

Project Gap / Contribution

This project reproduces and validates a software O-RU setup on a single host, with real OFH/eCPRI packet exchange between srsRAN DU and ProtO-RU.

System / Threat Model

System components:

- srsRAN gNB (CU + DU) - provides high-PHY, MAC, RRC, and NGAP
- ProtO-RU ru_emulator - provides low-PHY, IQ compression, and RF interface
- Open5GS 5G Core - AMF, SMF, UPF, NRF for subscriber and session management
- USRP B210 - SDR acting as the RF frontend driven by ProtO-RU
- Linux veth pair (veth_du ↔ veth_ru) - replaces the physical 10 GbE fronthaul link
- Software PTP (ptp4l + phc2sys on loopback) - replaces hardware grandmaster clock

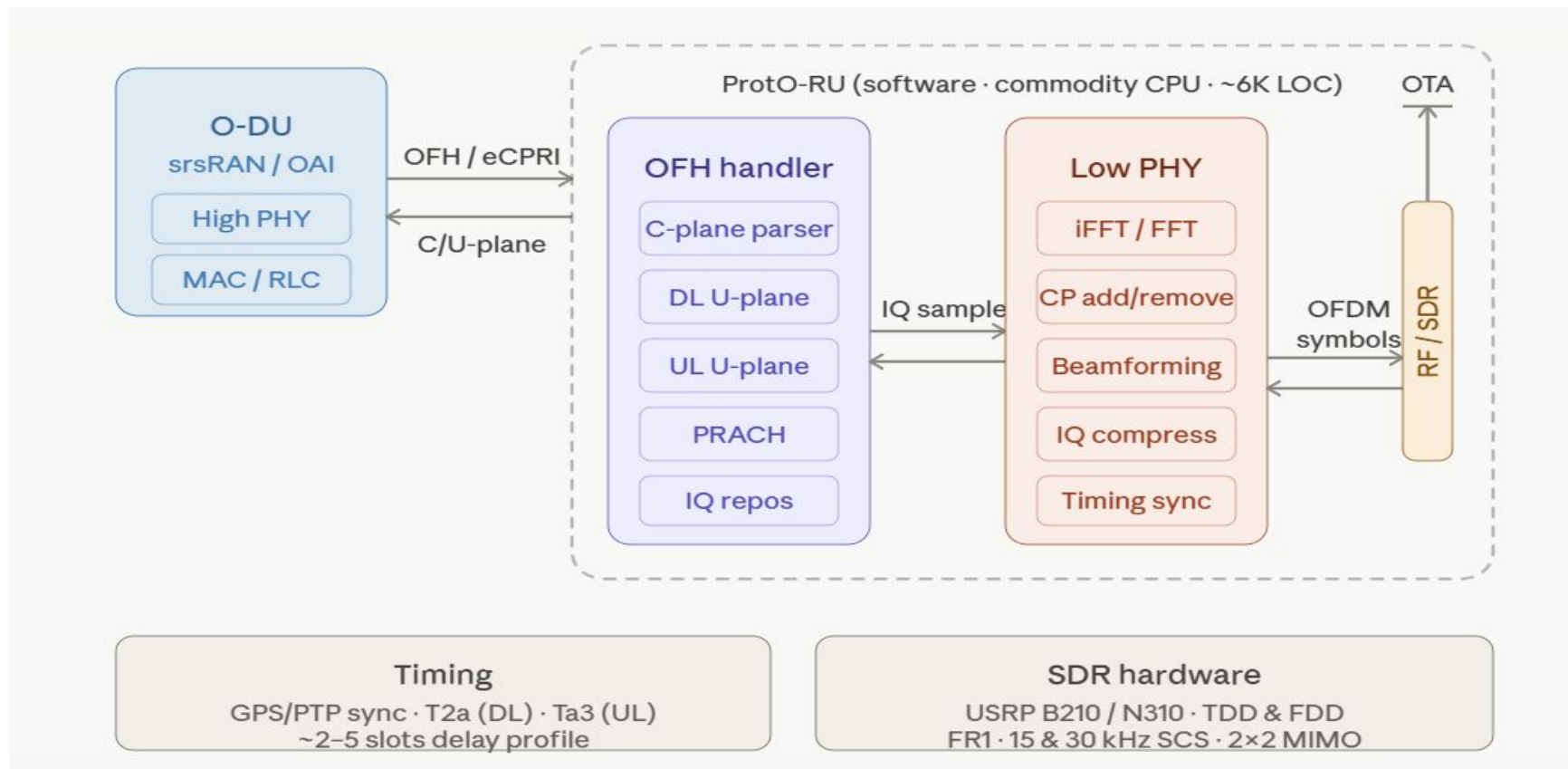
Key assumptions:

- All components run on a single host; physical separation between DU and RU is not modelled
- Software PTP is sufficient for the 100s-of-microsecond OFH timing windows used here

System Architecture

- System Model:
A general-purpose PC acting as the software processing node (O-RU), connected via network interface to a Distributed Unit (DU), and physically connected to an SDR frontend for RF transmission
- High-Level Design & Approach
 - Configure OS networking and time sync
 - Deploy ProtO-RU environment
 - Configure ru_emu.yml parameters
 - Bind UHD drivers to the SDR
 - Execute
- Data Flow: Open5GS ↔ gNB ↔ ProtO-RU over OFH/eCPRI on veth pair · ProtO-RU drives B210 over UHD/USB3 ↔ UE
- Contribution: Verified that the full ProtO-RU stack runs on a single host with virtual fronthaul · Fixed PLMN mismatch and OFH timing window misalignment to achieve 92% RX_ON_TIME

ProtO-RU architecture

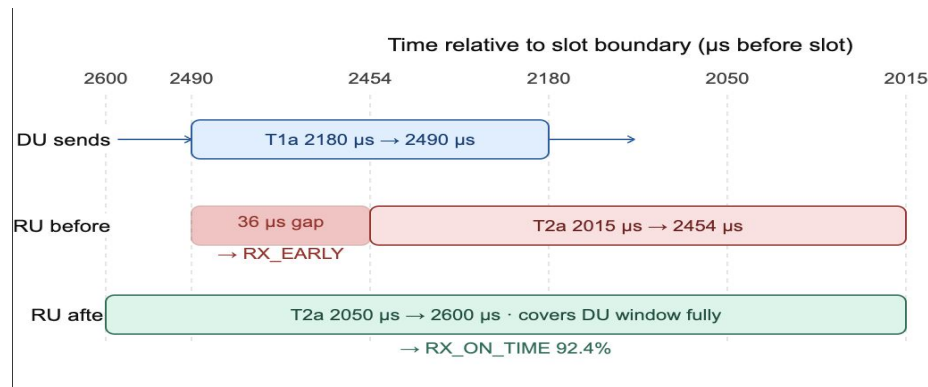


Key Algorithm/Logic

OFH timing window alignment (the core tuning problem):

The DU sends U-plane IQ samples within its T1a window. The RU must receive them within its T2a window. If the windows don't overlap, packets are dropped.

Time axis (relative to slot boundary):



Decision logic for packet classification at RU:

- Packet arrives before T2a_max_up AND after T2a_min_up → RX_ON_TIME
- Packet arrives after T2a_max_up → RX_EARLY (sent too early in pipeline)
- Packet arrives before T2a_min_up → RX_LATE (arrived too late)

Implementation Details

- Hardware Setup (if applicable):

Intel Xeon W-2133, 6 cores / 12 threads, 32 GB RAM, USRP B210, USB 3.0 connection, Ubuntu 24.04.4, Low-latency kernel 6.8.0-111-lowlatency

- Software & Tools:

Open5GS 2.7.7, srsRAN Project 24.10, ProtO-RU from NUS-CIR repository, UHD 4.6.0, linuxptp 4.0

Implementation Details

- Implementation:

What was built vs reused:

- Built: single-host configs, srsRAN T1a patch, veth setup, software PTP config
- Reused: srsRAN 24.10, ProtO-RU ru_emulator, Open5GS 2.7.7, UHD 4.6.0

Key challenges and fixes:

- Zero OFH packets despite gNB running → PLMN mismatch (001/01 vs 999/70) caused silent NGSetupFailure; fixed by aligning PLMN in gNB config
- 87% packets RX_EARLY → T2a_max_up (2454 μ s) narrower than T1a_max_up (2490 μ s); fixed by widening T2a window to 2600 μ s

Experimental Setup

- Setup

- Cell Configuration, Band: n78 TDD, DL ARFCN: 628032, Frequency: approximately 3420.48 MHz, Bandwidth: 20 MHz, SCS: 30 kHz, MIMO: 2x2, TDD pattern: DDDSU, Compression: BFP-9, PRACH config index: 159

- Reproducibility details

- Metrics used for Analysis

RX_TOTAL, RX_ON_TIME, RX_EARLY, RX_LATE, RX_SEQ_ERR, RX_CORRUPT,
TX_TOTAL C-plane RX_ON_TIME, Scheduler latency,

Baseline(s) for Comparison

- srsRAN Split-8 direct B210 setup

Results & Analysis

- Final runtime results:
 - OFH packet rate ~45,000 packets/sec,
 - RX_ON_TIME ~41,580 packets/sec,
 - RX_ON_TIME ratio ~92.4%,
 - RX_LATE ~30 packets/sec below 0.1%,
 - TX_TOTAL ~2,200 packets/sec, C-plane RX_ON_TIME ~3,200 packets/sec,
 - mean scheduler latency 5 μ s, scheduler errors 0.
- Analysis: The setup achieved bidirectional OFH/eCPRI packet exchange, zero RX_EARLY and zero sequence/corruption errors confirm that the DU and RU are exchanging valid fronthaul packets with aligned timing windows.

Limitations / Failure Cases

Core pinning not achieved

- Without pinning, OS scheduler causes thread jitter → 7.6% packets missed
- Result: 92% RX_ON_TIME instead of 99%+ from the paper
- Fix: clean stale cgroups, re-enable expert_execution, pin L1/OFH threads to dedicated cores

UE attach not achieved

- Second B210 opened fine — USB3, register loopback, PHY init all passed
- srsue searched for cell but never found it → no PRACH sent
- Fix: external clock reference (GPS-disciplined oscillator) or switch to OpenAirInterface NR-UE

Comparison with Objectives

Objective 1

Status (Completed/Partially Completed, Infeasible, etc)

Week1- Environment setup - Install Docker, UHD drivers, clone ProtO-RU

Completed

Week 2- Verify USRP B210 detection, PTP time sync configuration; build and launch srsRAN DU; first ProtO-RU startup attempt

Completed

Week 3- Debug fronthaul connectivity; validate eCPRI packet exchange; tune configuration parameters

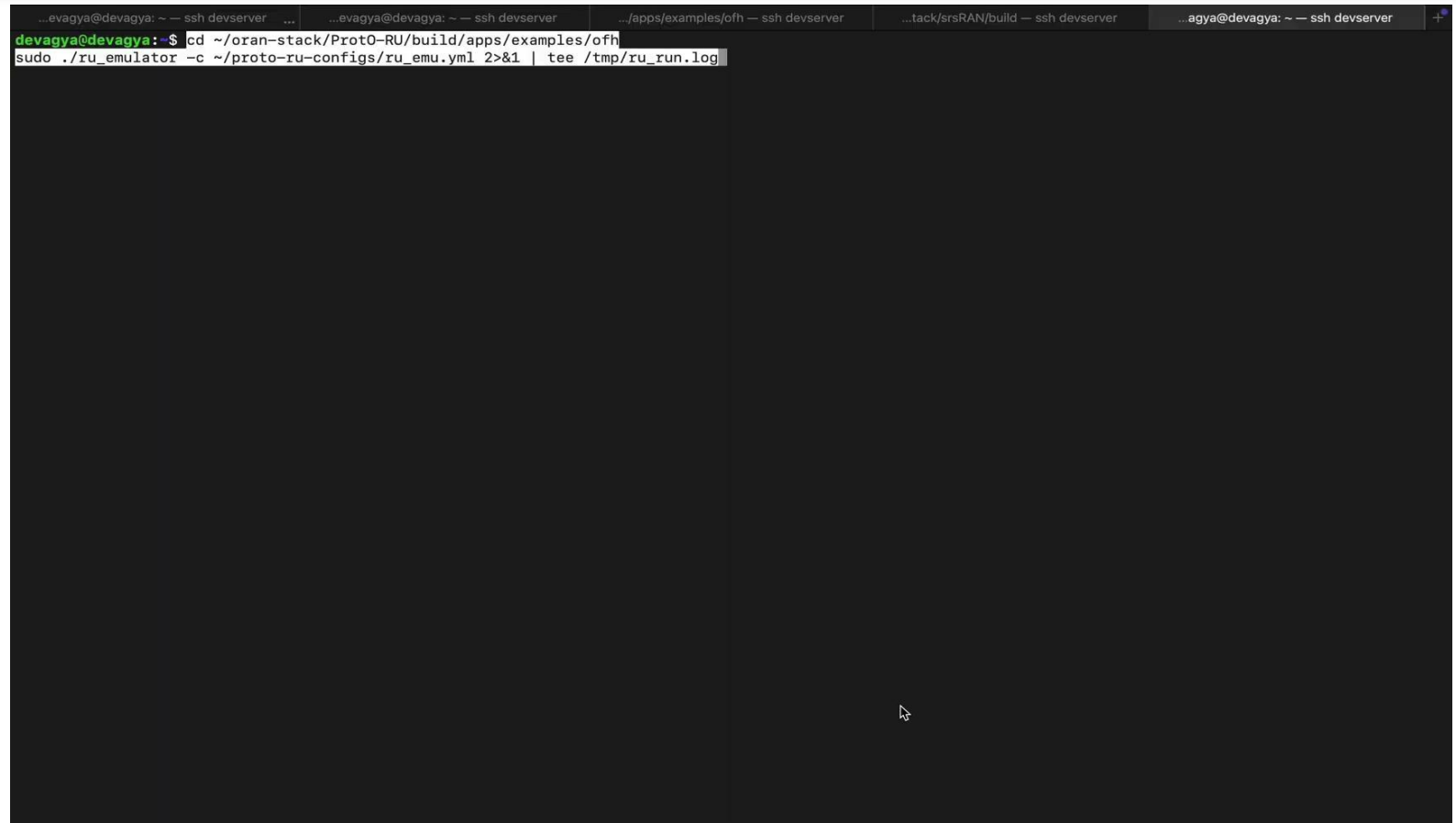
Completed

Week 4- End-to-end validation, documentation, final demo

Partially Completed(UE not connected)

Live Demo

- **Scenario:** The demo video shows ProtO-RU and the srsRAN gNB being started on the single-machine setup, followed by ProtO-RU receiving OFH/eCPRI packets from the DU with all packets classified as RX_ON_TIME, confirming successful and correctly timed fronthaul packet exchange.



```
devagya@devagya: ~ -- ssh devserver ... devagya@devagya: ~ -- ssh devserver .../apps/examples/ofh -- ssh devserver ...tack/srsRAN/build -- ssh devserver ...agya@devagya: ~ -- ssh devserver
devagya@devagya: ~$ cd ~/oran-stack/ProtO-RU/build/apps/examples/ofh
sudo ./ru_emulator -c ~/proto-ru-configs/ru_emu.yml 2>&1 | tee /tmp/ru_run.log
```

Work Distribution

Name	Core Responsibility	Key Tasks & Contributions
Devagya	End-to-end setup, integration, debugging, validation, and documentation	Installed and configured Open5GS • Built srsRAN 24.10 from source • Built ProtO-RU ru_emulator natively • Configured USRP B210 through UHD • Created single-machine veth fronthaul setup • Patched srsRAN T1a timing range • Tuned ProtO-RU T2a timing windows • Collected OFH packet and scheduler metrics • Prepared report and baseline comparison plan

Conclusions

Summary of key contributions:

- Full ProtO-RU + srsRAN + Open5GS stack built from source on a single machine - no Docker, no commercial RU
- Two non-obvious integration issues diagnosed and fixed: PLMN mismatch (blocked OFH entirely) and T2a timing window misalignment
- Achieved 92.4% RX_ON_TIME at ~45,000 OFH packets/sec with zero protocol errors
- Working configs, source patch, and evidence logs published on GitHub for reproducibility

Impact:

- Lowers the entry barrier to O-RAN Split-7.2 experimentation - single workstation + one B210 is enough to bring up the full stack
- Documents the integration pitfalls that the paper does not cover, making it easier for others to reproduce

Future work (if given 3 months):

- Re-enable core pinning to reach 99%+ RX_ON_TIME
- Add external 10 MHz clock reference to enable OTA UE attach with the second B210
- Run Split-8 vs Split-7.2 throughput and latency comparison with iperf3
- Move to two physical machines with a real 10 GbE link to fully replicate the paper's testbed
- Explore higher numerologies (60/120 kHz SCS) to test software PTP limits

References

- GitHub Repo Link: https://github.com/DevagyaYadav/ProtO-RU_Twin
- GitHub Repository: [NUS-CIR/ProtO-RU- https://github.com/NUS-CIR/ProtO-RU](https://github.com/NUS-CIR/ProtO-RU)
- Technical Report: "ProtO-RU: An O-RAN Split-7.2 Radio Unit using SDRs" - Z. Zhou et al., 2025. arXiv:2512.02398
- srsRAN Project Documentation - <https://docs.srsran.com>
- Open5GS Documentation - <https://open5gs.org/open5gs/docs/>
- linuxptp Documentation - <http://linuxptp.sourceforge.net>

Declaration on LLM Usage

- Claude (Anthropic) - Sonnet 4.6, ChatGPT 5.5
- Prompts
 - Explaining and modifying code snippets
 - Documentation of work
 - Debugging by giving logs
- Validation of scripts
 - Reading and understanding before applying
 - Build output was verified after each patch
 - Runtime behavior was checked
- How LLM generated code was documented:
 - Patches are documented with before/after comparison in the project GitHub repo
 - Each patch's purpose and effect is explained in the project README